Software Engineering
Lab 5 - Static Code Analysis

Name	: Rohan Suresh
SRN	: PES1UG24CS383
Section	: G
Assigned Repo	: https://github.com/SETAPESU26/L5_19_music_catalog
My Repo	: https://github.com/RohanMay-Suresh/SE-Labs-PES1UG24CS383 (Lab5 folder)

1. Introduction

Static code analysis means checking source code for bugs, security problems and style issues without actually running it. In this lab three Python tools were used on music_catalog.py, a small music catalog and playlist manager:

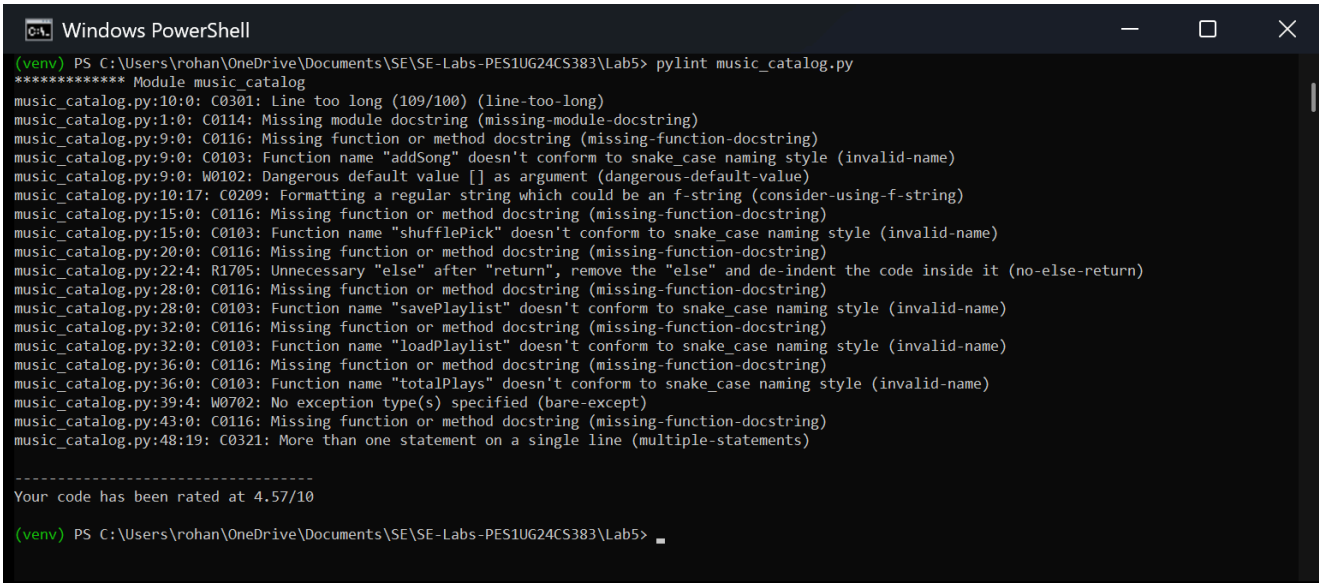
Pylint - checks code quality. It finds things like bad naming, missing docstrings, unused variables and bad practices, and gives the code a score out of 10.

Flake8 - checks the code against the PEP 8 style guide, like line length, spacing and multiple statements on one line.

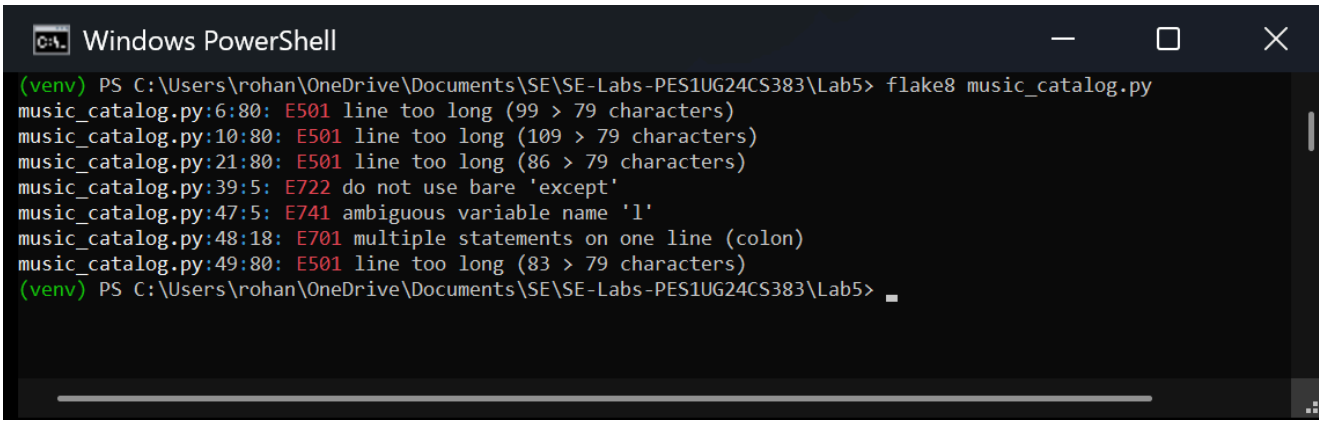
Bandit - checks for security problems, like SQL injection, unsafe use of pickle and insecure random numbers.

2. Issues Found (Before Fixing)

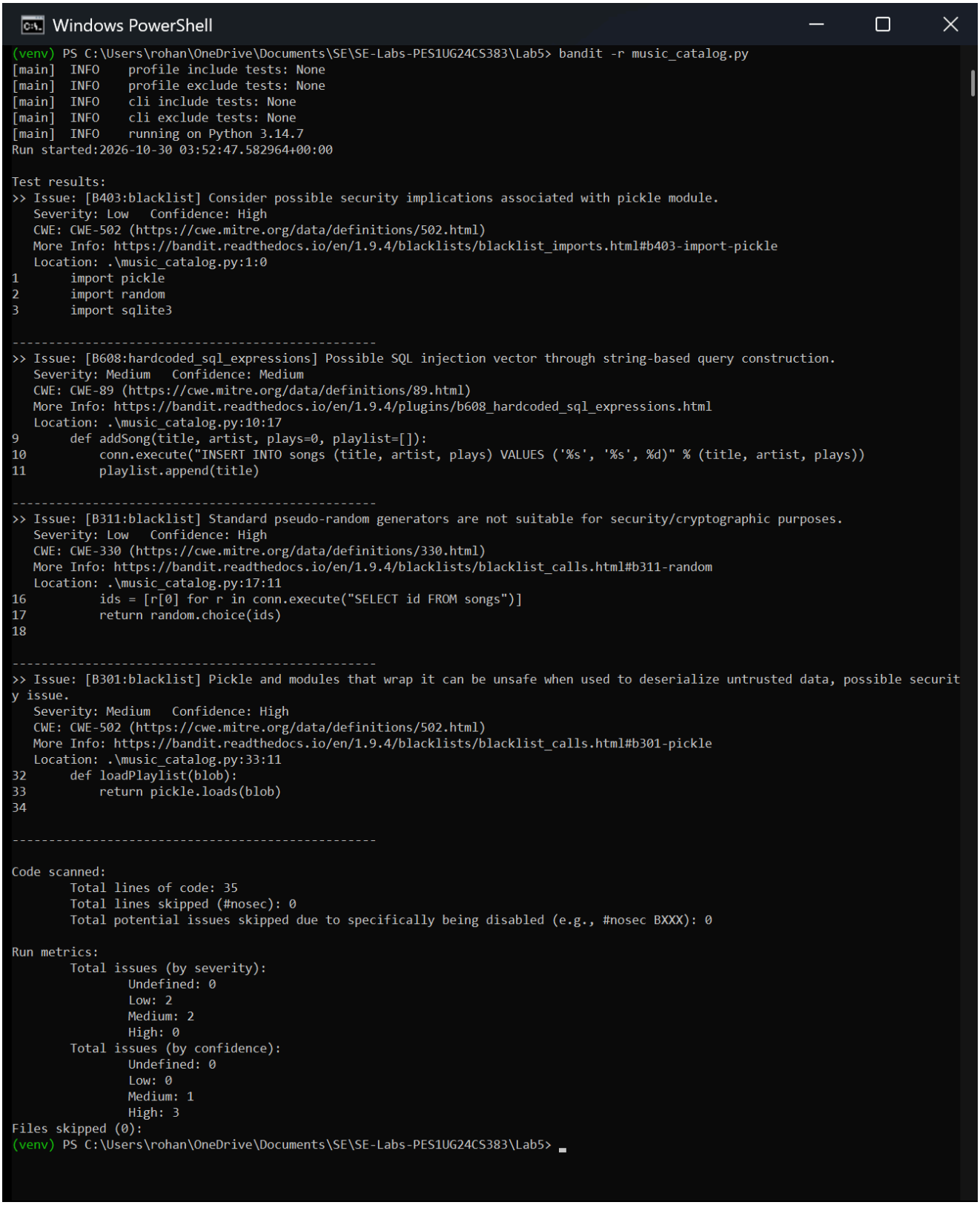
Pylint



Flake8



Bandit


Windows PowerShell
(venv) PS C:\Users\rohan\OneDrive\Documents\SE\SE-Labs-PES1UG24CS383\Lab5> bandit -r music_catalog.py
[main] INFO profile include tests: None
[main] INFO profile exclude tests: None
[main] INFO cli include tests: None
[main] INFO cli exclude tests: None
[main] INFO running on Python 3.14.7
Run started:2026-10-30 03:52:47.582964+00:00
Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
Severity: Low Confidence: High
CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
Location: .\music_catalog.py:1:0
1 import pickle
2 import random
3 import sqlite3
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
Severity: Medium Confidence: Medium
CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
Location: .\music_catalog.py:10:17
9 def addSong(title, artist, plays=0, playlist=[]):
10 conn.execute("INSERT INTO songs (title, artist, plays) VALUES ('%s', '%s', %d)" % (title, artist, plays))
11 playlist.append(title)
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
Severity: Low Confidence: High
CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
Location: .\music_catalog.py:17:11
16 ids = [r[0] for r in conn.execute("SELECT id FROM songs")]
17 return random.choice(ids)
18
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, possible securit
y issue.
Severity: Medium Confidence: High
CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
Location: .\music_catalog.py:33:11
32 def loadPlaylist(blob):
33 return pickle.loads(blob)
34
Code scanned:
Total lines of code: 35
Total lines skipped (#nosec): 0
Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
Run metrics:
Total issues (by severity):
Undefined: 0
Low: 2
Medium: 2
High: 0
Total issues (by confidence):
Undefined: 0
Low: 0
Medium: 1
High: 3
Files skipped (0):
(venv) PS C:\Users\rohan\OneDrive\Documents\SE\SE-Labs-PES1UG24CS383\Lab5>

3. Issue Identification Table

Line numbers are from the original file.

Issue	Type	Line(s)	Description	Fix Approach
SQL injection (B608)	Security - Bandit, Medium	10	the insert query was made with % formatting so title and artist go straight into the sql. a title like "Don't Stop" crashes the program	used ? placeholders and passed the values as a tuple
pickle.loads (B301)	Security - Bandit, Medium	33	pickle can run code when loading data from an untrusted source	used json.dumps / json.loads instead since the playlist is just a list of strings
Mutable default arg (W0102)	Bug - Pylint	9	playlist=[] is shared between calls so old songs show up in new playlists	changed default to None and made a new list inside the function
Bare except (W0702, E722)	Bug - Pylint/Flake8	39	catches every exception and hides actual errors	changed it to except sqlite3.Error
import pickle (B403)	Security - Bandit, Low	1	importing pickle is flagged	removed it with the json change
random.choice (B311)	Security - Bandit, Low	17	random is not meant for anything security related	changed to secrets.choice
Function names (C0103)	Style - Pylint	9, 15, 28, 32, 36	function names were camelCase	renamed to snake_case (add_song, shuffle_pick etc)
Missing docstrings (C0114, C0116)	Style - Pylint	whole file	no module or function docstrings	added docstrings
else after return (R1705)	Style - Pylint	22-25	else is not needed after a return	removed the else
Multiple statements on one line (C0321, E701)	Style - Pylint/Flake8	48	if and print on the same line	put the print on its own line
Line too long (E501)	Style - Flake8	6, 10, 21, 49	lines longer than 79 characters	split the lines
Ambiguous variable name (E741)	Style - Flake8	47	variable called l	renamed it to songs

The fixes were made in 3 separate commits: fixed bandit issues, fixed pylint issues and fixed flake8 issues.

4. Tool Outputs Before and After

pylint_report.txt (before)

```
************* Module music_catalog
music_catalog.py:10:0: C0301: Line too long (109/100) (line-too-long)
music_catalog.py:1:0: C0114: Missing module docstring (missing-module-docstring)
music_catalog.py:9:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:9:0: C0103: Function name "addSong" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:9:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
music_catalog.py:10:17: C0209: Formatting a regular string which could be an f-string (consider-using-f-string)
music_catalog.py:15:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:15:0: C0103: Function name "shufflePick" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:20:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:22:4: R1705: Unnecessary "else" after "return", remove the "else" and de-indent the code inside it
(no-else-return)
music_catalog.py:28:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:28:0: C0103: Function name "savePlaylist" doesn't conform to snake_case naming style (invalid-nam
e)
music_catalog.py:32:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:32:0: C0103: Function name "loadPlaylist" doesn't conform to snake_case naming style (invalid-nam
e)
music_catalog.py:36:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:36:0: C0103: Function name "totalPlays" doesn't conform to snake_case naming style (invalid-name)
music_catalog.py:39:4: W0702: No exception type(s) specified (bare-except)
music_catalog.py:43:0: C0116: Missing function or method docstring (missing-function-docstring)
music_catalog.py:48:19: C0321: More than one statement on a single line (multiple-statements)

-----------------------------------
Your code has been rated at 4.57/10
```

pylint_report.txt (after)

```
-------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

flake8_report.txt (before)

```
music_catalog.py:6:80: E501 line too long (99 > 79 characters)
music_catalog.py:10:80: E501 line too long (109 > 79 characters)
music_catalog.py:21:80: E501 line too long (86 > 79 characters)
music_catalog.py:39:5: E722 do not use bare 'except'
music_catalog.py:47:5: E741 ambiguous variable name 'l'
music_catalog.py:48:18: E701 multiple statements on one line (colon)
music_catalog.py:49:80: E501 line too long (83 > 79 characters)
```

flake8_report.txt (after)

(empty file - flake8 prints nothing when there are no issues)

bandit_report.txt (before)

```
Run started:2026-10-30 03:50:08.936253+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
   Location: .\music_catalog.py:1:0
1   import pickle
2   import random
3   import sqlite3

--------------------------------------------------
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Medium
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: .\music_catalog.py:10:17
9   def addSong(title, artist, plays=0, playlist=[]):
10      conn.execute("INSERT INTO songs (title, artist, plays) VALUES ('%s', '%s', %d)" % (title, artist, plays))
11      playlist.append(title)

--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\music_catalog.py:17:11
16      ids = [r[0] for r in conn.execute("SELECT id FROM songs")]
17      return random.choice(ids)
18

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted data, p
ossible security issue.
   Severity: Medium   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: .\music_catalog.py:33:11
32  def loadPlaylist(blob):
33      return pickle.loads(blob)
34

--------------------------------------------------

Code scanned:
    Total lines of code: 35
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 2
        Medium: 2
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 1
        High: 3
Files skipped (0):
```

bandit_report.txt (after)

```
Run started:2026-10-30 03:53:37.831258+00:00

Test results:
    No issues identified.

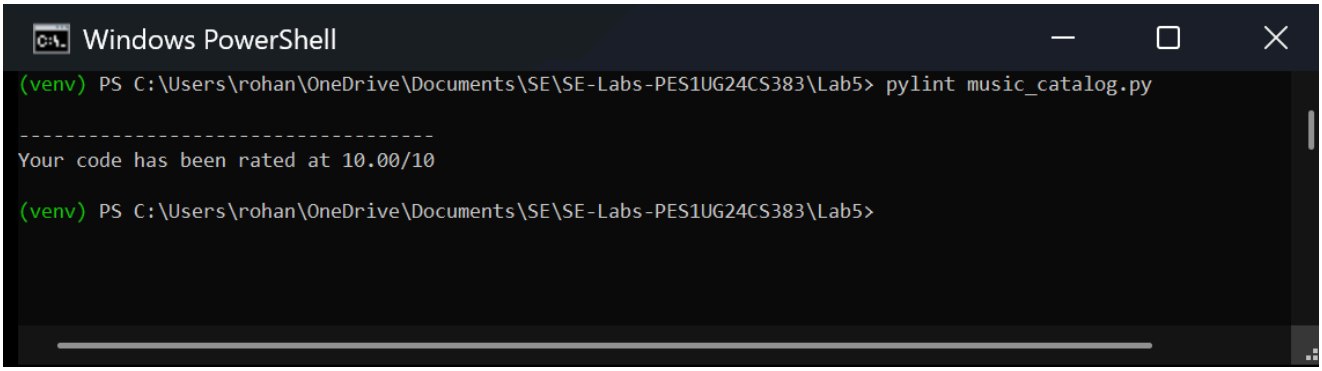
Code scanned:
    Total lines of code: 56
    Total lines skipped (#nosec): 0
```

```
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

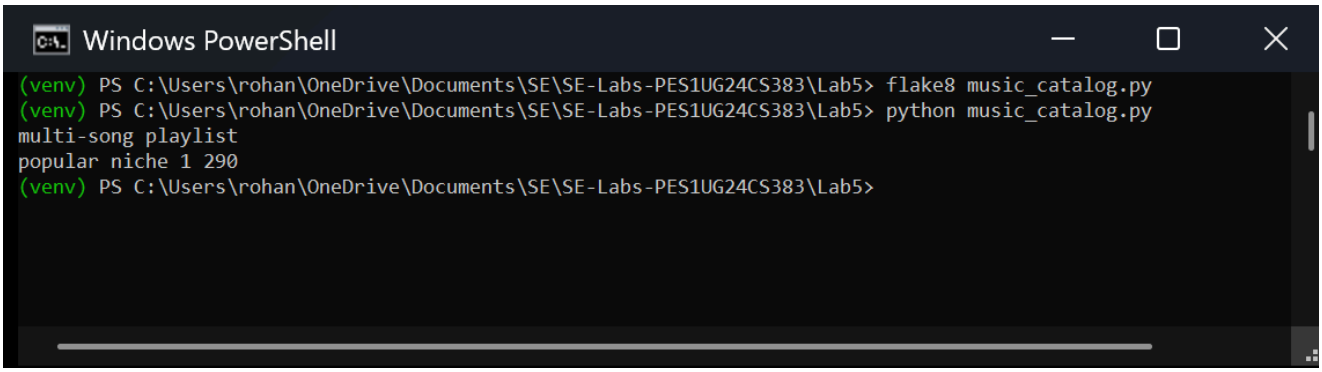
Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
```

5. Results After Fixing

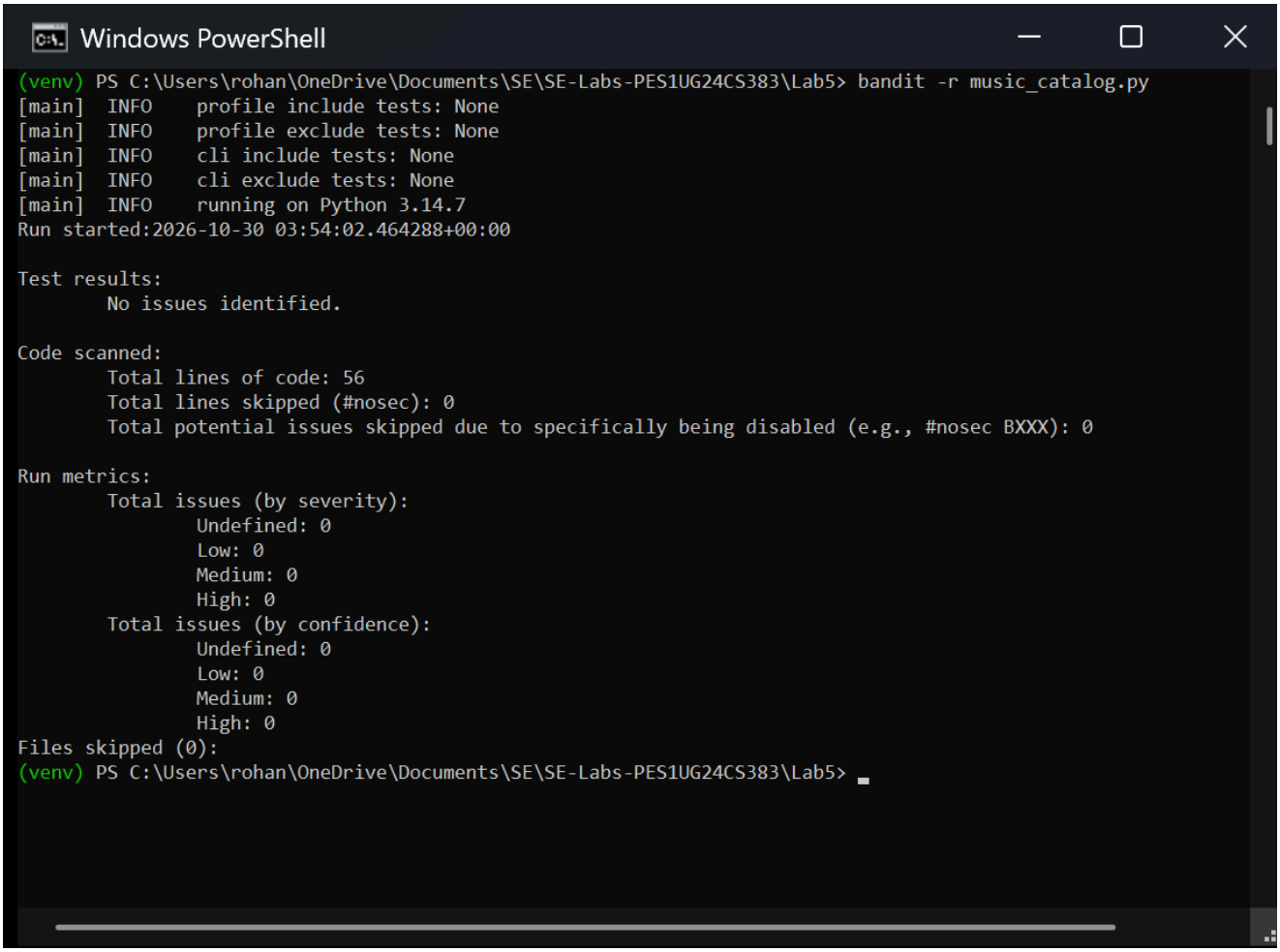
Pylint



Flake8 (and the program still running)



Bandit



Pylint went from 4.57/10 to 10.00/10, Bandit went from 4 issues to 0 and Flake8 went from 7 issues to 0. The program gives the same output as before.

6. Reflection

1. Which issues were the easiest to fix, and which were the hardest? Why?

The style issues were the easiest, like renaming functions to snake_case, adding docstrings and splitting long lines. They don't change how the code works. The hardest one was replacing pickle because I had to make sure saving and loading the playlist still worked after the change. json worked since the playlist is only a list of strings. The mutable default argument also took a bit of time to understand.

2. Did the static analysis tools report any false positives? If so, describe one example.

Yes, Bandit flagged random.choice (B311) even though it is only used to pick a random song, which has nothing to do with security. I still changed it to secrets.choice since we had to handle all the Bandit issues, but normally I would just leave it.

3. How would you integrate static analysis tools into your actual software development workflow?

I would run flake8 and pylint before every commit, either from the editor or with a pre-commit hook. In a team project they can also be added to CI (like GitHub Actions) so the checks run on every push and the build fails if there are serious Bandit issues. Fixing a few warnings each time is easier than fixing everything at the end.

4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

The program doesn't crash anymore when a song title has an apostrophe, and loading a playlist can't run random code now. The mutable default bug where old songs were showing up in new playlists is also fixed. The code is easier to read with proper names and docstrings. Pylint went from 4.57 to 10.00 and Bandit and Flake8 don't show any issues.